

reproducing_np_finite_bug_random_seed_100

April 22, 2025

```
[26]: import sys
from IPython.display import display, Javascript

def restart_kernel():
    """Restart the Jupyter Notebook kernel to reflect changes in modules and
    ↪ packages."""
    display(Javascript("Jupyter.notebook.kernel.restart()"))
    print("Kernel is restarting...")

restart_kernel()

import numpy as np
import torch
import tensorly as tl
from tensorly.decomposition import parafac
import sparse
import pickle
import os
import numpy.random as rn
import scipy.stats as st
from tensorly.cp_tensor import CPTensor

from bptf import BPTF as BPTF
import bptf
```

<IPython.core.display.Javascript object>

Kernel is restarting...

1 Helper functions

```
[27]: def generate(shp=(30, 30, 20, 10), K=5, alpha=0.1, beta=0.1):
    """Generate a count tensor from the BPTF model.

    PARAMS:
    shp -- (tuple) shape of the generated count tensor
    K -- (int) number of latent components
    alpha -- (float) shape parameter of gamma prior over factors
```

```

    beta -- (float) rate parameter of gamma prior over factors

    RETURNS:
    Mu -- (np.ndarray) true Poisson rates
    Y -- (np.ndarray) generated count tensor
    """
    Theta_DK_M = [rn.gamma(alpha, 1./beta, size=(D, K)) for D in shp]
    Mu = tl.cp_to_tensor(CPTensor((None, Theta_DK_M)))
    assert Mu.shape == shp
    Y = rn.poisson(Mu)
    return Mu, Y

```

2 Load data

```

[28]: use_existing_data = False

rn.seed(100)

if use_existing_data:
    assert os.path.exists('sptensor.pkl'), 'No such file.'
    with open('sptensor.pkl', 'rb') as f:
        data = pickle.load(f)
        data = data[:, :, :, :12, :]
else:
    data = generate(shp=(200, 200, 20, 12, 3), K=10)[1]
    data = sparse.COO(data)

```

3 Building mask

```

[29]: mask = np.zeros(data.shape)
    # april of GDELT is set to missing
    mask[:, :, :, 3, 1] = 1
    # diagonals are set to missing
    mask[np.eye(mask.shape[0]).astype(bool)] = 1
    mask = sparse.COO(mask.astype(np.int64))

```

4 Fit model

```

[30]: n_components = 10
    max_iter = 100

    BPTF_model = BPTF(data_shape=data.shape, n_components=n_components)
    BPTF_model.fit(data, mask = mask, max_iter = max_iter, verbose=False,
        ↪missing_val=1, tol=1e-10)

```

```

30%|          | 30/100 [04:40<10:54, 9.35s/it]

```

```

-----
AssertionError                                Traceback (most recent call last)
Cell In[30], line 5
      2 max_iter = 100
      4 BPTF_model = BPTF(data_shape=data.shape, n_components=n_components)
----> 5
      ↳ BPTF_model.fit(data, mask = mask, max_iter = max_iter, verbose=False, missing_val=1, tol=1e-4)

File ~\OneDrive - The University of Chicago\UChicago_MS_Stat\
↳ aaron_schein\bptf_new\bptf\src\bptf\bptf.py:267, in BPTF.fit(self, data, mask, missing_val, **kwargs)
      263     assert is_binary(mask)
      265     self._init()
--> 267     self._update(data, mask=mask, missing_val=missing_val, **kwargs)
      268     return self

File ~\OneDrive - The University of Chicago\UChicago_MS_Stat\
↳ aaron_schein\bptf_new\bptf\src\bptf\bptf.py:242, in BPTF._update(self, data, mask, modes, **kwargs)
      235     if kwargs.get('verbose', True):
      236         print ('ITERATION %d:\t'\
      237               'Time: %f\t'\
      238               'Objective: %.2f\t'\
      239               'Change: %.5e\t'\
      240               '% (itn+1, e, bound, delta))
--> 242     assert delta >= 0.0
      243     curr_elbo = bound
      244     if delta < kwargs.get('tol', 1e-4):

AssertionError:

```